

The Secret Language of LLMs



Now let's understand the input of an LLM. Does it understand only English? But we see that it also responds in different languages. We know programming languages are used because computers don't understand English, that's why we have specific languages to make computers understand, like C and C++. So what about LLMs?

Computers don't understand English words, that we know 100%. If we are saying, **"These notes are awesome,"** this sentence cannot be directly understood by an LLM. What an LLM understands is numbers.

Let's see how. So when we say **"These notes are awesome,"** it gets broken down into different words, like:

`These` `notes` `are` `awesome`

And these words are converted into numbers, and an array of numbers is passed to the LLM, like:

`[78, 12, 23, 433]`

When these sentences are broken into words or pieces, they are called **tokens**.

The number associated with a token is called the **token ID**, and the sequence of tokens is what the LLM understands.

But the question comes: if an LLM knows the numbers, then how does it predict the next word? So instead of predicting the next word directly, the LLM predicts the next **token ID**, which then maps back to a token. This is an informal way to understand the secret language of LLMs.

Breaking a sentence or unit of text into tokens is done by a **tokenizer**.

There are multiple types of tokenizers available that companies use. Different models and different types of tasks can use different tokenization methods. This is one of the most important parts of LLMs, used during training and inferencing. You can learn more about tokenizers online.

One more thing: it doesn't guarantee that one word becomes one token. It is possible that:

Awesome

is converted into two tokens, like:

awe

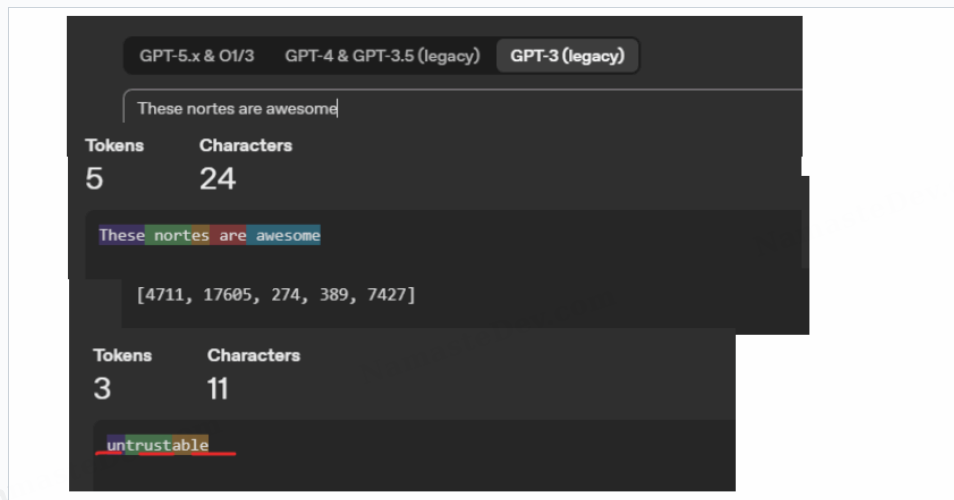
some

So, one word is not always one token.

Okay, okay, so much theory. Let's see these things live on websites. These are two websites you can check to see how it works:

<https://platform.openai.com/tokenizer> <https://tiktokenizer.vercel.app/>

Now here I have put our statement. Here is how the output comes:



On the website, you can see you can use different tokenizers, and that generates different token IDs and breaks the text in different ways. But the question comes: what are these numbers, and why are these random numbers?

Actually, when a word is converted into smaller parts, it is called **subword tokenization**. The question is, why do we need this?

Subword tokenization depends on the tokenizer and the vocabulary it has learned. If a word or piece of a word is very common, it may become one token, but a larger or less common word can become multiple subword tokens.

Like, see the above screenshot: 'untrustable' becomes 3 tokens because parts like 'un', 'trust', and 'able' can represent meaningful patterns and can be reused in other words. This helps the model handle many different words without needing every possible word to exist as a separate token.

I think now you are able to understand it.

But I saw something where I was confused: 'undone' becomes only one token. I was expecting it to break into two tokens, like 'un' + 'done'. So why did that happen?

The answer is that **tokenization is not always based on grammar or the meaning of a word**. The tokenizer does not necessarily split a word into its meaningful parts. If 'undone' already exists as a common token in that tokenizer's vocabulary, it can be represented by a single token. The same word can also be split differently by another tokenizer.

So, **one word is not always one token, and one prefix + word is also not guaranteed to become multiple tokens. It depends on the tokenizer vocabulary and how that tokenizer was trained.**

Vocabulary & Token ID

Every tokenizer has a vocabulary. This is the way each token gets matched to a token ID. And how does that happen? Using the vocabulary mapping.

So, in easy words:

A tokenizer's **vocabulary** is the fixed list of all unique words, subwords, or characters that an AI model or text processor recognizes and can convert into numbers.

So, many companies use different types of tokenizers, as we discussed. OpenAI models use different 'Byte Pair Encoding (BPE)' tokenizers, categorized by their different vocabulary sizes. Meta Llama uses 'SentencePiece-BPE' frameworks. Read more online; it's not possible to write everything here.

So, understand that the model has a vocabulary where tokens like 'un', 'trust', and 'able' are mapped to numbers using the vocabulary.

Byte Pair Encoding (BPE)

This is one type of tokenizer, and this is used by OpenAI. Here, what we want to understand is, suppose there is one word, 'low', and it has '15' as the number assigned. Now it can be used for 'low', 'lower', and 'lowest'. The same goes for 'new', 'newer', and 'newest', and also for 'un', 'unbreakable', 'untrustable', 'unmanageable', etc.

So, using BPE, we can achieve this. Each token can be represented using bytes. You know a byte can be represented by 8 bits, I mean '0' and '1'. So these combinations of bits represent numerical values. The tokenizer starts from smaller byte-level units and learns which neighbouring pairs commonly occur together. These pairs can then be merged into new tokens, and the vocabulary size increases.

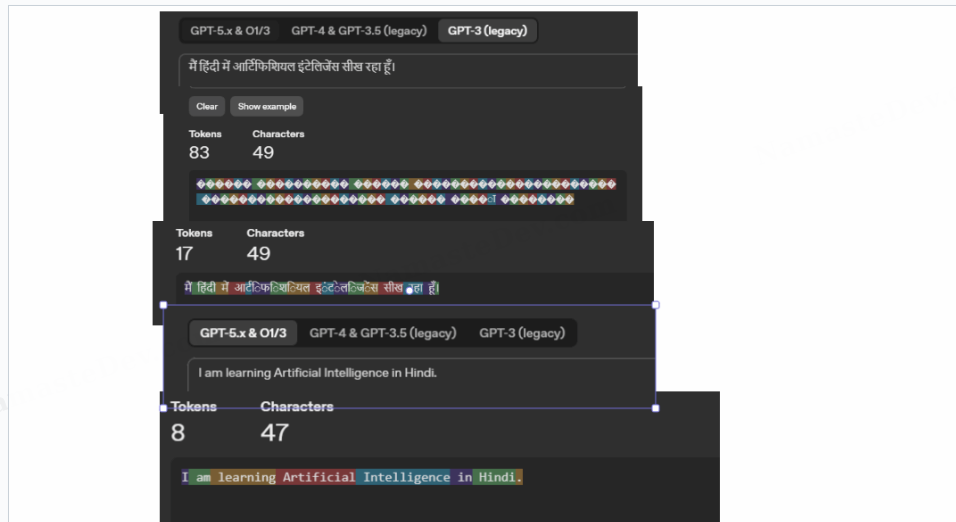
So it's all about **merging neighbouring pairs**. Common combinations can become a single token, which is why the same smaller pieces can be reused across many words.

There are different tokenizer methods also used, like 'WordPiece' and 'Unigram'. Please read about those online and just understand vocab size make tokenization very good and efficient

English vs Other Languages

I am learning Artificial Intelligence , आइए आर्टिफिशियल इंटेलिजेंस सीख रहा हूँ।

Or if we write in Hinglish: **Mai Artificial Intelligence seekh raha hu** or mix of that, so don't think it is all tokenized the same way because the meaning is the same. No, there is no meaning of words that LLM understands, and tokenization happens based on the text. There is no emotion, so different languages can have different token IDs. Remember it.



Now you can see in the image, with the older version, it was broken into the same line into so many tokens, but in the modern tokenizer version, it uses fewer tokens. So vocabulary size increases, and new languages are more efficiently supported by OpenAI. So vocabulary is the main factor in understanding how a model handles different languages.

But English is more efficient. See the above image: the same thing in English takes fewer tokens, but Hinglish is more costly in terms of tokens.

But it's crazy that it understands Hinglish so well, because in Hindi we write a word in different ways, like **main, mai, maii**, and it can be written in multiple ways.

Tokenization fertility

Tokenization fertility is **the average number of tiny text pieces (tokens) an AI creates for every single word**, meaning how many tokens are produced from a word.

Lower fertility means the AI uses fewer pieces to represent a word. This makes the AI faster, cheaper to run, and able to process more text at once. [1, 2]

Higher fertility means the AI chops words into lots of tiny fragments. This uses more tokens, increases the cost, and fills up the context window faster, especially in some non-English languages.

How do LLMs understand emojis, code, special characters, capital vs small-case letters? Let's understand.

Emojis and characters are also mapped in the vocabulary, so everything is properly engineered, and that gives us the related result. Spaces also change the token ID, so it's all about mapping.

Check any tokenization playground, like the OpenAI tokenizer or any other tokenizer available online. You will see that adding or removing a space, using curly braces, adding dashes, or removing dashes can all change the tokenization and token count.

Each text pattern has its own token ID and mapping.

Okay, there are some special tokens also added by the LLM when we do a prompt. Those are from the system instructions. What are these system instructions? With each model, all we provide system instructions.

We will learn later about system tokens, the start of the message, the end of the message, and how these are based on the thing we are writing, like a text document, etc.

A **system instruction** is a hidden rulebook that tells an AI how to behave, while a **user instruction** is the specific question or task you type in.

User instructions are **simple, clear steps that tell a person how to use a product, app, or tool**.

So, the way these system and user prompts are passed to an LLM is different for each LLM model and company.

Context window



You remember when you chat with an AI assistant and you come back after 7 days, it also remembers what you chatted about before 7 days, and you keep the project and all the chats under that. This is all possible because it keeps the context, or we can say memory, of the chat.

So, each LLM has a **context window**. In that window, it remembers your chat. A context window is the amount of tokenized information a model can process within a request or active generation context.

In easy words, a **context window is the short-term working memory of an AI, measuring the maximum amount of text it can "see" and remember at any single moment**.

But context includes the **system instructions, documents, text, tool outputs, everything**, not just what you are typing.

The context window is shared between what you send and what the model generates.

So what happens if the context window overflows?

So, first of all, to avoid it, **truncate your input**, summarize previous messages, don't put the entire thing as one input, and always ask for the relevant things that you need.

Split the work into different chats, module-wise, not everything in one chat. **Work A on Chat A, Work B in Chat B**. Do not assume AI always remembers your requests, which are very old. It remembers the summary, but only while the context is live!

Context doesn't mean chat history, so never think that.

Remember, a longer prompt does not always give you a better result. **Tokens directly affect API cost.**

There are also tools available where you can input your big prompt, and they will provide you with a polished prompt. That will make you more efficient and cost-effective.

So, just list some misconceptions people have:

- **One token is equal to one word:** This is wrong.
- **Everyone uses the same tokenizer:** This is also wrong.
- **Token ID represents meaning:** Answer is no.
- **One visible emoji is one token:** Answer is no.
- **A larger vocabulary is always better:** No. (If I say yes, I need to search why.)
- **Larger context means perfect memory:** Not necessarily.
- **More tokens always lead to better results:** No.

Ok so that's all for this one, will see you in next one, Bye Bye !



LEARN BY VIDEO

Explore courses on NamasteDev.com

Hands-on video lessons from Akshay Saini's ecosystem.

Start learning ↗